

FINAL PROJECT	
Course Code: CPE 009B	Program: Computer Engineering
Course Title: Object Oriented Programming 2	Date Performed: September 12, 2026
Section: CPE21S2	Date Submitted:
Group Member(s): THOMAS JOSEPH ASTUDILLO JUAN PAULO LARA CARLO MENESES BENJ ARIEL RATCHO JOHREL LOUIE SILAO CHRISTIAN LEE BANDIGAN	Instructor: Engr. Noel Jason Lusung
A. Instruction	
Simple Chat Application (Username, chat threading w/ emoji) Tasks: - Optional (login: username/ pw, can access multiple accounts) - Username (avatar: optional) - Chat thread (date/time) - Limit text per chat 300char - Send Emoji - Compliment color palette (logo), send button, edit (optional) - Local (multiple users allowed)	
Materials & Equipment - Personal Computer (Windows 10/11) - Python Version 3.13 – 3.14 - Visual Studio Code / Spyder IDE - PyQt6	
Discussion Algorithm - Menu Interface - Load system menu - User Authentication - Check if the user wants to log in or switch accounts. - If logging in. Prompt user for username and password. - Check if the user’s username and password is correct. - If the user has successfully logged in. Proceed to the chat menu interface. - Chat Interface & Threads - Load existing chat messages.	

- **Preparing and Validating Messages**

1. Input message with or without emojis.
2. Check character limit
3. If the user's inputted text length is ≤ 300 characters. Can send messages.
4. If the user's inputted text length is > 300 , send out a warning prompt to alert the user.

- **Sending Messages**

1. When clicking the send button:
 - a. Prints out the date and time
 - b. Saves the messages to a local file.
 - c. Update the chat thread.

Pseudocode

Procedures

Code

Menu Interface

Python

```
import sys
import os
import ctypes
from PyQt6.QtWidgets import QWidget, QApplication, QLabel
from PyQt6.QtGui import QIcon, QMovie, QGuiApplication
try:
    ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID('lab4.pyqt6.gui')
except Exception:
    pass
class App(QWidget):
    def center_window(self):
        screen_geometry = QGuiApplication.primaryScreen().availableGeometry()
        x = (screen_geometry.width() - self.width) // 2 + screen_geometry.left()
        y = (screen_geometry.height() - self.height) // 2 + screen_geometry.top()
        self.move(x, y)
    def __init__(self):
        super().__init__()
        self.title = "PyQt Labels"
        self.x = 200
        self.y = 200
        self.width = 300
        self.height = 300
        self.initUI()
    def initUI(self):
```

```

self.setWindowTitle(self.title)
self.resize(self.width, self.height)
self.center_window()
script_dir = os.path.dirname(os.path.abspath(__file__))
# Window Icon
icon_path = os.path.join(script_dir, 'pythonico.ico')
self.setWindowIcon(QIcon(icon_path))
# GIF Integration
gif_path = os.path.join(script_dir, 'giphy.gif')
self.gif_label = QLabel(self)
self.gif_label.setGeometry(100, 160, 100, 100)
self.movie = QMovie(gif_path)
self.gif_label.setMovie(self.movie)
self.gif_label.setScaledContents(True)
self.movie.start()
# Labels
self.textboxlbl = QLabel("Hello World!", self)
self.textboxlbl.move(110, 50)
self.sublbl = QLabel("This program is written in Visual Studio", self)
self.sublbl.move(35, 90)
self.show()
if __name__ == '__main__':
app = QApplication(sys.argv)
ex = App()
sys.exit(app.exec())

```

User Authentication

Python

```

import sys
import os
from PyQt6.QtWidgets import QWidget, QLabel, QLineEdit, QPushButton, QMessageBox,
QApplication
from PyQt6.QtGui import QIcon, QFont, QtGuiApplication, QMovie
class RegistrationWindow(QWidget):
def __init__(self):
super().__init__()
self.title = "Account Registration System"
self.width = 420
self.height = 460
self.initUI()
def initUI(self):
self.setWindowTitle(self.title)
# Base path for relative resources
script_dir = os.path.dirname(os.path.abspath(__file__))
icon_path = os.path.join(script_dir, 'pythonico.ico')
self.setWindowIcon(QIcon(icon_path))
self.resize(self.width, self.height)
self.center_window()
# Program Title
title_label = QLabel("Account Registration", self)
title_font = QFont('Arial', 14)

```

```

title_font.setWeight(QFont.Weight.Bold)
title_label.setFont(title_font)
title_label.move(110, 15)
# GIF Integration
gif_path = os.path.join(script_dir, "200w.gif")
self.gif_label = QLabel(self)
self.gif_label.setGeometry(160, 50, 100, 100)
# Ensure the file exists before attempting to load
if os.path.exists(gif_path):
    self.movie = QMovie(gif_path)
    self.gif_label.setMovie(self.movie)
    self.gif_label.setScaledContents(True)
    self.movie.start()
else:
    self.gif_label.setText("GIF Not Found")
# Form Absolute Positioning
top_start = 165
row_height = 35
label_x = 40
field_x = 160
field_width = 200
field_height = 25
fields = [
    ("First Name:", "txt_first_name", False),
    ("Last Name:", "txt_last_name", False),
    ("Username:", "txt_username", False),
    ("Password:", "txt_password", True),
    ("Email Address:", "txt_email", False),
    ("Contact Number:", "txt_contact", False)
]
self.inputs = {}
for idx, (label_text, field_name, is_password) in enumerate(fields):
    current_y = top_start + (idx * row_height)
    lbl = QLabel(label_text, self)
    lbl.move(label_x, current_y + 3)
    txt = QLineEdit(self)
    txt.setGeometry(field_x, current_y, field_width, field_height)
    if is_password:
        txt.setEchoMode(QLineEdit.EchoMode.Password)
    self.inputs[field_name] = txt
    buttons_y = top_start + (len(fields) * row_height) + 15
    self.btn_submit = QPushButton("Submit", self)
    self.btn_submit.setGeometry(80, buttons_y, 110, 32)
    self.btn_submit.clicked.connect(self.submit_form)
    self.btn_clear = QPushButton("Clear", self)
    Python
    self.btn_clear.setGeometry(220, buttons_y, 110, 32)
    self.btn_clear.clicked.connect(self.clear_form)
def center_window(self):
    screen = QApplication.primaryScreen().geometry()
    x = (screen.width() - self.width) // 2
    y = (screen.height() - self.height) // 2
    self.move(x, y)
def submit_form(self):
    first_name = self.inputs['txt_first_name'].text()
    if not first_name:

```

```
QMessageBox.warning(self, "Warning", "Please fill in required fields!")
else:
QMessageBox.information(self, "Success", "Account registered successfully!")
def clear_form(self):
for txt in self.inputs.values():
txt.clear()
if __name__ == "__main__":
app = QApplication(sys.argv)
window = RegistrationWindow()
window.show()
sys.exit(app.exec())
```

B. Output(s) and Observation(s)

GitHub Repository

<https://github.com/jplara-cpe2025/CPE-009B-CPE21S2-FINAL-PROJECT-GROUP2-2526>

C. Conclusion & Lessons Learned